

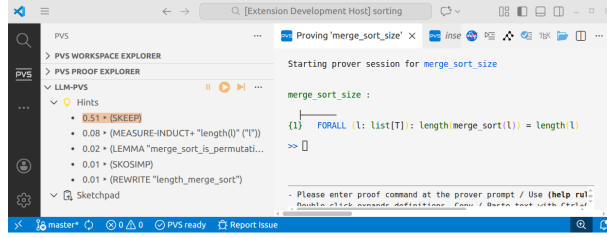
1 LLM-PVS Integration Framework

This section describes how users can interact with LLM-PVS thanks to the integration into the VSCode-PVS extension, one of main contributions of this work. Figure 1 shows a guiding example where the first five consecutive steps of the proof of the theorem `merge_sort_size` are depicted. This theorem states that the function implementing merge-sort preserves the length of the input list. Because of space limitations, this Section focuses only on recommendations of proof commands. It is worth noting that the proof-command recommendations may include commands related to the inclusion of lemmas, but these are independent from the lemma-suggestion feature described in the previous section.

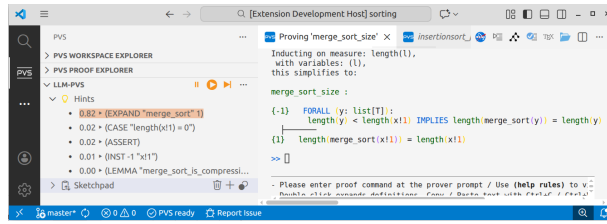
Figure 1a depicts how the initial sequent of the proof is presented to the user in VSCode-PVS. The current sequent is displayed in the proof console (on the right-hand side), and the model recommendations are shown in the LLM-PVS panel (on the left-hand side) ranked by model certainty. It should not be surprising that the top-ranked recommendation for the initial sequent points to the Skolemization of the target formula, via the `SKEEP` command with 51% of certainty, since most of the proofs in the library start in this way. In second place, LLM-PVS proposes to apply an induction scheme over the list 1 using the `MEASURE-INDUCT+` command, which is a direct way to prove this property over lists. Notably, the command arguments in the recommendation are correct in content and position. The following recommendation in the rank is to introduce a lemma via the `LEMMA` command. The proposed lemma could actually be useful in this proof, but it is declared after the one being currently proved and thus it is not applicable. The `SKOSIMP` command in the fourth place is another flavor of Skolemization. Finally, the fifth recommendation is to rewrite the formula in the sequent using the lemma `length_merge_sort`, but this is a hallucination, since there is no lemma with that name declared in the library.

In the recommendations generated for this sequent, the manifestation of the *data-bias* issue can be seen. For instance, there are two main ways to perform induction in PVS: `MEASURE-INDUCT` and `INDUCT`. While the latter command usually produces simpler proofs for data structures such as lists, the model suggested the former, reproducing the preference of the authors of the proofs from the training library. This is because, like any other deep learning model, LLM-PVS tries to extract hidden patterns and reproduce them, thereby repeating the bias from the training data and actually, in this case, favoring the attempt of more complicated proofs. Similarly, both `SKEEP` and `SKOSIMP` are commands for Skolemizing; there is a clear preference for `SKEEP` in the training data, reflected in the model certainty: 51% versus 1%, respectively.

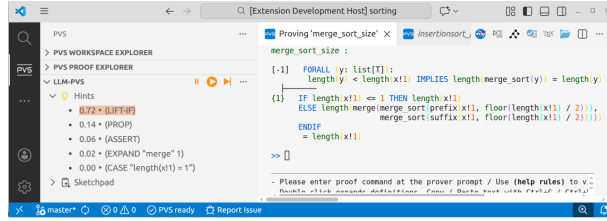
The user can tap one recommendation, and the command is sent to the proof console. After performing the induction, the screen shown in Figure 1b is presented. In order to use the inductive hypothesis added to the sequent as formula -1 , it is necessary to expose the recursive nature of merge-sort by expanding its definition. The model is highly confident (82%) about this expansion, specifically on Formula 1. The second recommendation proposes analyzing the base case, where the list is empty. The `ASSERT` command is a powerful mechanism that



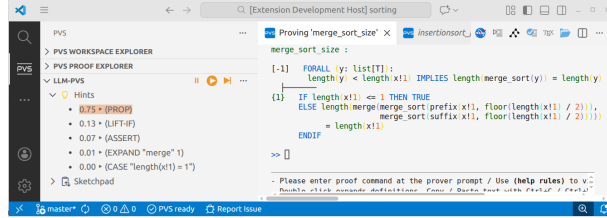
(a)



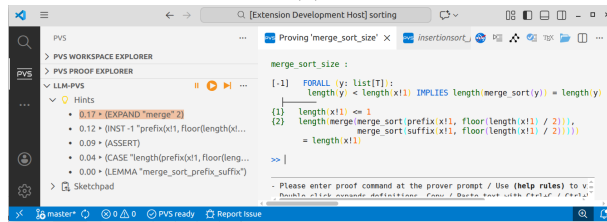
(b)



(c)



(d)



(e)

Fig.1: Interaction with LLM-PVS during the proof of the lemma `merge_sort_size` in VSCode-PVS. The depicted Visual Studio Code screenshots are used with permission from Microsoft.

simplifies the goal using decision procedures and automatic rewriting. Next in order of certainty, LLM-PVS recommends to instantiate the inductive hypothesis using the `INST` command, although the suggested instantiation is a dead end. As for the initial sequent, the fifth recommendation is the introduction of a non-existent lemma. Note that in both hallucinations, the model certainty for such suggestions was close to zero.

Figure 1c shows the result of expanding the `merge_sort` function. The model is highly confident (72%) in lifting the conditionals (`LIFT-IF`), a step needed to work on each of the cases in the definition of the function. The other suggestions are also applicable. For instance, the expansion of `merge` enables the instantiation of the inductive hypothesis.

The sequent in the Figure 1d is the result of applying the `LIFT-IF` command. The first recommendation, `PROP`, would produce a split in the proof following the branches of the conditional. The model also suggests `LIFT-IF` with low certainty (13%), the same command used before. As shown in the inference phase (Section ??), the model input is solely the current sequent, and the model extracts complex statistics of the input, which does not provide a global proof context, leading to repetitions of the same recommendation in sequential steps. In this case, it is possible that the model learned the relation between conditional keywords in sequent and the `LIFT-IF` application.

Figure 1e shows the sequent after applying the first recommendation. At this point, the **Trained LLM** suggests two top options with near the same certainty; either expand the `merge` definition, or instantiate the inductive hypothesis with the correct list. Both alternatives are possible steps to advance the proof. Indeed, this approach *assists* the user during the interactive proof section. The responsibility to *prove* remains on the user, checking the applicability of the recommendations and choosing the more convenient proof plan.